FODS ASSIGNMENT 3

Name – Atharva Shivaji Dabkekar
Roll no. – 2090139
PRN - 1251090357
Division – S.Y A
Batch – 2
DATE – 28/07/26 TIME – 8:00 TO 10:00

QUESTION) Practical 3: Multidimensional Arrays and Sparse Matrix

Implement two-dimensional arrays with row-major and column-major address calculations. Represent sparse matrices and perform simple and fast transpose operations.

ANSWER)

Row Major Order: A method of storing a matrix in memory where elements are stored row by row, from left to right.

Column Major Order: A method of storing a matrix in memory where elements are stored column by column, from top to bottom.

Sparse Matrix: A matrix in which most of the elements are zero, and only the non-zero elements are stored to save memory.

CODE)

```
#include <iostream>

using namespace std;

struct Sparse

{

   int row;

   int col;

   int value;

};

int main()
```

```
{
    int r, c;
    cout << "Enter number of rows and columns: ";
    cin >> r >> c;
    int a[10][10];
    cout << "Enter matrix elements:\n";
    for(int i = 0; i < r; i++)
    {
        for(int j = 0; j < c; j++)
        {
            cin >> a[i][j];
        }
    }
    // Row-major and Column-major Address Calculation
    int x, y;
    cout << "\nEnter row and column to find address: ";
    cin >> x >> y;
    int base = 1000;
    int size = 4;
    int rowMajor = base + ((x * c) + y) * size;
    int columnMajor = base + ((y * r) + x) * size;
    cout << "\nRow Major Address = " << rowMajor << endl;
    cout << "Column Major Address = " << columnMajor << endl;
    // Sparse Matrix Representation
    Sparse s[100];
    int k = 1;
    s[0].row = r;
    s[0].col = c;
    s[0].value = 0;
```

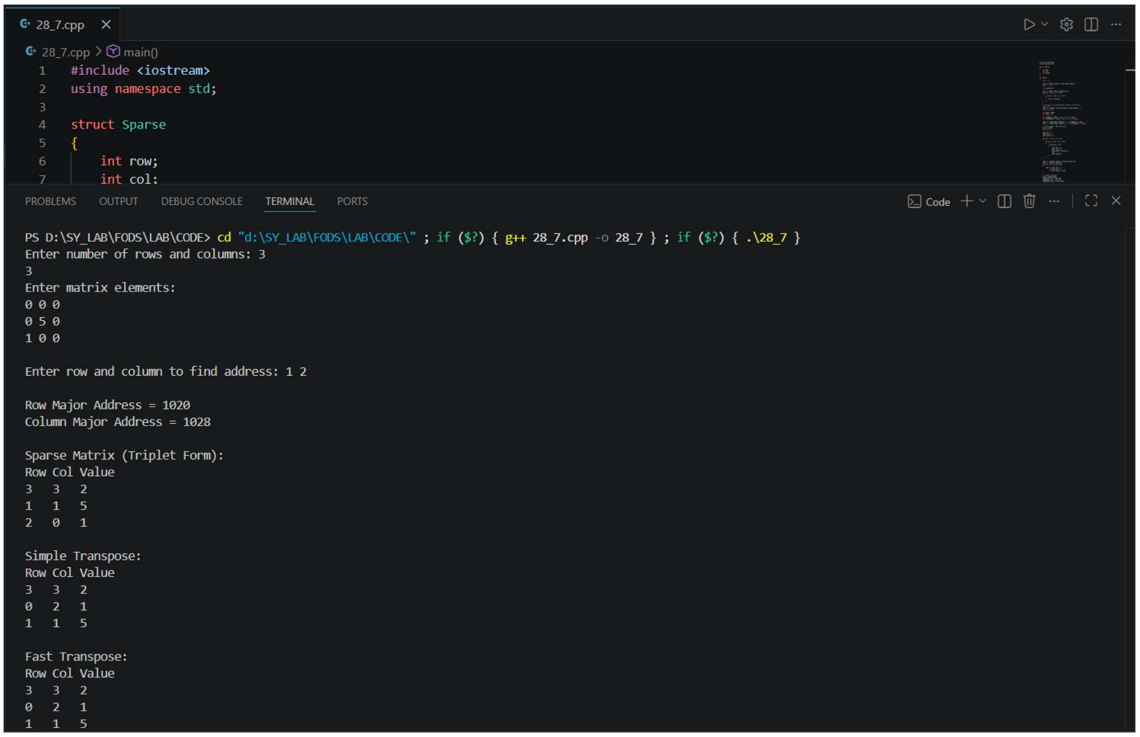
```
for(int i = 0; i < r; i++)
{
   for(int j = 0; j < c; j++)
   {
      if(a[i][j] != 0)
      {
         s[k].row = i;
         s[k].col = j;
         s[k].value = a[i][j];
         k++;
         s[0].value++;
      }
   }
}
cout << "\nSparse Matrix (Triplet Form):\n";
cout << "Row Col Value\n";
for(int i = 0; i < k; i++)
{
   cout << s[i].row << "   "
        << s[i].col << "   "
        << s[i].value << endl;
}

// Simple Transpose
Sparse simple[100];
simple[0].row = s[0].col;
simple[0].col = s[0].row;
simple[0].value = s[0].value;
int p = 1;
```

```
for(int i = 0; i < c; i++)
{
   for(int j = 1; j < k; j++)
   {
      if(s[j].col == i)
      {
         simple[p].row = s[j].col;
         simple[p].col = s[j].row;
         simple[p].value = s[j].value;
         p++;
      }
   }
}
cout << "\nSimple Transpose:\n";
cout << "Row Col Value\n";
for(int i = 0; i < p; i++)
{
   cout << simple[i].row << "   "
        << simple[i].col << "   "
        << simple[i].value << endl;
}
// Fast Transpose
Sparse fast[100];
int total = s[0].value;
fast[0].row = s[0].col;
fast[0].col = s[0].row;
fast[0].value = total;
int rowTerms[10] = {0};
int startPos[10];
```

```
    for(int i = 1; i <= total; i++)
        rowTerms[s[i].col]++;
    startPos[0] = 1;
    for(int i = 1; i < c; i++)
        startPos[i] = startPos[i - 1] + rowTerms[i - 1];
    for(int i = 1; i <= total; i++)
    {
        int pos = startPos[s[i].col];
        fast[pos].row = s[i].col;
        fast[pos].col = s[i].row;
        fast[pos].value = s[i].value;
        startPos[s[i].col]++;
    }
    cout << "\nFast Transpose:\n";
    cout << "Row Col Value\n";
    for(int i = 0; i <= total; i++)
    {
        cout << fast[i].row << "   "
             << fast[i].col << "   "
             << fast[i].value << endl;
    }
   return 0;
}
```

OUTPUT)


28_7.cpp
main()
#include <iostream>
using namespace std;
struct Sparse
{
int row;
int col;
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
Code
PS D:\SY_LAB\FODS\LAB\CODE> cd "d:\SY_LAB\FODS\LAB\CODE\" ; if ($?) { g++ 28_7.cpp -o 28_7 } ; if ($?) { .\28_7 }
Enter number of rows and columns: 3
3
Enter matrix elements:
0 0 0
0 5 0
1 0 0
Enter row and column to find address: 1 2
Row Major Address = 1020
Column Major Address = 1028
Sparse Matrix (Triplet Form):
Row Col Value
3 3 2
1 1 5
2 0 1
Simple Transpose:
Row Col Value
3 3 2
0 2 1
1 1 5
Fast Transpose:
Row Col Value
3 3 2
0 2 1
1 1 5